MetroInsights - FastAPI Interview Preparation Playbook

Current-edition project guide | 24 October 2026

This guide describes the repository after the backend migration from Flask to FastAPI. Treat it as the current framework reference; earlier study sheets may still describe Flask. The FastAPI source and deployment configuration are in place, but the current local virtual environment does not yet contain FastAPI/Uvicorn, so a runtime smoke check remains pending dependency installation.

1. Project in a minute

MetroInsights is a DMRC planning prototype for exploring station-level passenger-demand scenarios. A user chooses a station, date, time, and optionally a corridor. The app estimates demand with a saved scikit-learn model, adds calendar and service-hours context, calculates a configurable train-frequency/capacity scenario, and estimates network travel times. It presents the results in a browser dashboard with charts and station search.

It is a simulation and planning demonstration, not a live DMRC operations system. The training targets are synthetic, operational assumptions are configurable project assumptions, and the travel graph depends on project topology and coordinate quality. The model metrics measure agreement with a synthetic holdout, not real-world accuracy.

Current verified project figures

Item	Current value	How to describe it accurately
Modeled station records	243	Repository station roster, not a claim that every current DMRC physical location is officially represented
Modelled hours per day	19	05:00 through 23:00 inclusive
Synthetic training/evaluation rows	3,361,176	243 x 728 dates x 19 hourly records
Chronological holdout rows	438,615	Synthetic records on/after the 2026-10-01 split
Model features	23	Calendar, time, station/line/typology, and prior-history features
Selected model	HistGradientBoostingRegressor	Compared with historical-mean and Ridge baselines
Holdout MAE	26.58 passengers/hour	Synthetic targets only
Holdout RMSE	51.10 passengers/hour	Synthetic targets only
Holdout R-squared	0.9946	Synthetic targets only; does not establish external validity
Reachable destinations from Rajiv Chowk	242	Current graph/UI result; topology and some segment times are modeled

These values are read from `data/cleaned/stations_cleaned.csv`, `data/synthetic/dmrc_calibrated_footfall.csv`, and `models/evaluation_metrics.json`. Recheck them after data regeneration or retraining.

30-second introduction

“I built MetroInsights, a FastAPI-based Delhi Metro planning prototype. It combines a scikit-learn passenger-demand model trained on synthetic scenarios with calendar context, configurable train-service recommendations, and Dijkstra travel-time estimates. The browser dashboard lets a user compare station, date, time, and corridor scenarios. I label the estimates as simulations because the training targets are synthetic and the operational assumptions are not verified DMRC data.”

Two-minute introduction

“MetroInsights explores how estimated passenger demand changes by station, date, and hour, and how that estimate relates to a possible service plan. The web layer is FastAPI and Jinja2, with JavaScript calling a JSON endpoint for live dashboard updates. The prediction layer loads a saved HistGradientBoostingRegressor and its feature metadata, then builds the 23 expected inputs for a chosen scenario. Separate modules add calendar and operating-hours context, calculate frequency and capacity recommendations from configured assumptions, and find weighted routes on a line-expanded metro graph. The offline pipeline prepares station data, generates more than 3.3 million synthetic hourly examples, compares model baselines on a chronological holdout, and writes model artifacts. The model's strong holdout metrics describe fit to generated labels only. The next major step for real planning use would be observed ridership data and verified operational inputs.”

Five-minute narrative outline

1. Problem and user: explore station/time demand scenarios for planning discussions.
2. User flow: station/date/time/corridor selection, forecast response, charts/KPIs, recommendation, travel times.
3. Architecture: FastAPI + Jinja/static browser UI; model/scheduling/calendar/graph modules; offline data pipeline.

4. ML: 23 features, chronological split, historical-mean and Ridge comparisons; synthetic metrics with explicit limitations.
5. Engineering judgment: keep training offline, load fitted artifacts at runtime, make assumptions visible, and do not claim live or validated DMRC outputs.

2. Architecture and request flow

```
Browser HTML/CSS/JavaScript
  |-- GET / and /dashboard --------------------> Jinja2 templates
  |-- POST /api/predict (JSON) ----------------> FastAPI Request parsing
  |                                               -> thread-pool inference function
  |                                               -> calendar + service-hours logic
  |                                               -> saved model + schedule optimizer
  |                                               -> weighted graph shortest paths
  |<-- JSON response --------------------------- update dashboard charts and cards
  |-- POST /get_footfall (form) ---------------> legacy server-rendered report page
  |-- GET /api/stations, /api/model_metrics ----> JSON metadata and metrics

Offline pipeline: clean station/operations inputs -> generate synthetic rows -> train,
evaluate, and save model + metadata. The web process serves those saved artifacts;
it does not train on the large synthetic CSV for each request.
```

Routes and their purpose

Route	Methods	Purpose
`/`	GET	Landing page with scenario preview
`/dashboard`	GET	Dashboard shell and initial station metadata
`/api/predict`	GET, POST	Forecast, context, service recommendation, profile, and travel-time JSON
`/api/stations`	GET	Station list with line/interchange/coordinate metadata
`/api/model_metrics`	GET	Saved evaluation summary for display
`/get_footfall`	POST	Form-driven server-rendered scenario report
`/api/docs`	GET	FastAPI-generated OpenAPI documentation UI

Startup sequence

1. Uvicorn imports `run:app` in deployment, or `run.py` starts Uvicorn for local use.
2. Importing the `app` package constructs a FastAPI instance and mounts `app/static`.
3. `app.routes` is imported to register endpoints. It reads station metadata and builds the route graph at module load.
4. `app.forecasting` loads the joblib model and metadata and station table at module load. Operations settings are also read by the scheduling module.
5. Requests use the in-memory artifacts. Missing files, incompatible joblib/scikit-learn versions, or malformed input files can prevent startup.

This is a small single-process app design. It is simple to reason about but import-time initialization means each worker needs the required files and may load its own copies in memory.

3. Repository map: what matters most

Path	Responsibility	Interview focus
`run.py`	Local Uvicorn launch, `PORT`, optional reload setting	ASGI server entry point
`Procfile`	Render/Uvicorn start command	Deployment command and `$PORT`
`requirements.txt`	FastAPI, Uvicorn, multipart, Jinja, ML/data libraries	Reproducible dependency installation
`app/__init__.py`	FastAPI app, metadata, static mount, route import	App construction and route registration
`app/routes.py`	Page/API endpoints, request parsing, orchestration	Main backend request flow
`app/forecasting.py`	Artifact loading, feature assembly, prediction, 19-hour profile	Model serving and feature parity
`app/scheduling_optimizer.py`	Line split, peak logic, headway, fleet and capacity scenario	Deterministic business logic and assumptions
`app/calendar_context.py`	Workday/occasion context	Calendar features and provenance
`app/operating_hours.py`	Service-open predicate	Closed-hour behavior and boundaries
`app/travel_graph.py`	Line-expanded weighted graph and shortest paths	Dijkstra, transfer penalties, coordinate fallbacks
`app/network_topology.py`	Project-defined line sequences, interchange links, aliases	Network completeness and source confidence
`app/templates/`	Jinja2 landing/dashboard/report pages	Server-rendered HTML and template context
`app/static/js/dashboard.js`	JSON fetch and dashboard rendering	Frontend/backend response contract
`pipeline/01_audit_and_clean_data.py`	Cleans station and operations inputs	Data-quality assumptions
`pipeline/02_generate_calibrated_dataset.py`	Creates synthetic date/hour/station records	Simulation generation, not observed data
`pipeline/03_train_and_evaluate_forecaster.py`	Feature build, chronological split, baseline/model evaluation, serialization	Leakage control and evaluation limitations
`models/`	Saved estimator, metadata, metrics	Runtime artifacts and version compatibility
`data/cleaned/`	Station roster and operational assumptions	Runtime inputs; coordinates/topology need review

Important dependencies and saved inputs

The request path relies on compact cleaned CSV/JSON files and joblib artifacts. The multi-million-row synthetic CSV supports pipeline training/retraining and is not needed for ordinary inference when compatible model artifacts are present. The current code has no relational database, ORM, authentication layer, message queue, or external live ridership API.

4. FastAPI implementation deep dive

App construction: `app/__init__.py`

`FastAPI(...)` creates the ASGI application and metadata used by OpenAPI. `app.mount("/static", StaticFiles(...), name="static")` serves CSS and JavaScript. Importing `app.routes` registers the route functions. `BASE_DIR` is resolved from the source file path so model/data paths do not depend on the caller's working directory.

The app is a module-level instance rather than an application factory. An app factory and explicit lifespan initialization could improve test isolation and startup error reporting if the app grows.

Templating and static URLs

`Jinja2Templates` renders `landing.html`, `index.html`, and `result.html`. FastAPI route handlers pass the request and a context dictionary to `TemplateResponse`. Templates construct static asset URLs through `request.url_for('static', path=...)`, which resolves against the named static mount.

JSON API: `/api/predict`

The browser sends `{station, date, time, line}` as JSON. The async endpoint reads and validates that the body is a JSON object, or accepts query parameters for GET. It then uses Starlette's `run_in_threadpool` to call synchronous model/scheduling/graph work. This keeps CPU-bound and blocking Python work off the event loop; making the calculation `async` would not itself make scikit-learn or Pandas nonblocking.

The synchronous worker then:

1. Normalizes the selected station and parses date/time, returning a 400 JSON error for invalid date/time syntax.
2. Gets calendar and service-open context and station/line metadata.
3. Predicts the selected hour and the full 19-point operating-day profile when service is open; closed times are labeled and their selected-hour demand is zeroed.
4. Calls the scheduling optimizer with demand, line metadata, weekday/workday, and service status.
5. Chooses a line-specific recommendation or aggregates interchange demand/capacity for station overview.
6. Calculates route times with Dijkstra over the line-expanded graph.
7. Adds explicit data-provenance labels and returns a JSON response.

The route currently uses a raw `Request` rather than a Pydantic request model. This keeps GET and POST forms flexible, but the generated OpenAPI docs do not provide a strongly typed request schema for prediction inputs. A small Pydantic model and explicit response model would be a useful FastAPI-specific improvement.

Threading and async boundary

The prediction endpoint is `async` only for request-body reading. Its model and graph work is passed to `run_in_threadpool`. The report route similarly awaits form parsing and offloads synchronous computation/render preparation. Normal `def` routes such as dashboard, stations, and metrics are run by FastAPI in a thread pool. Avoid placing CPU-heavy synchronous logic directly inside an `async def` route because it blocks the event loop.

Error and edge behavior to understand

- Invalid date or time syntax returns HTTP 400 with a user-readable message.
- A non-object JSON body returns HTTP 400.
- A closed hour should show no modeled service and no selected-hour demand.
- Unknown station handling depends on downstream `get_station_info`; the API should be hardened to return a deliberate 404/422 rather than an unhandled server error.

- The form endpoint supports the current browser form. `python-multipart` is required by Starlette form parsing and is included in requirements.
- There is no custom global exception handler or structured logging layer in this migration.

5. Data and model: know what the scores mean

Generated dataset

The generator builds one row per station/date/hour for 728 dates and 19 hours. At 243 station records, that yields 3,361,176 rows. Pandas handles tables and feature frames; NumPy is used for numerical operations and seeded randomness. The rows come from a project-authored simulation of station, time, calendar, and history patterns.

Features and prediction

The 23-feature set includes hour/day/month/calendar fields, cyclical encodings, station/line/typology encodings, interchange/line-count values, and three historical-demand aggregates. Runtime code loads persisted metadata and uses the recorded `feature_cols` order to assemble the model input. Keeping training and inference feature definitions in sync is essential: column order, encodings, defaults, and history availability must match.

The model is a fitted `HistGradientBoostingRegressor`. Pipeline 03 also computes historical-mean and Ridge baselines. The chronological split date is 2026-10-01; holdout records total 438,615. The final saved summary reports MAE 26.58, RMSE 51.10, R-squared 0.9946, and MAPE 5.18% for the selected model.

Correct interpretation

The chronology and prior-date history features help avoid direct future-label leakage in the generated split. They do not remove simulation bias: the train and test targets are still generated by the same synthetic process. The high R-squared is evidence of learning that generated process, not evidence the model predicts actual Delhi Metro passenger counts with 99.46% accuracy. If observed counts become available, evaluate on them with a time-based holdout and document station/time coverage.

Baseline context

The historical `(station, day-of-week, hour)` mean baseline reports MAE 60.25 and Ridge reports 53.67 on the same synthetic test split. Comparing baselines is stronger than presenting one model score alone, but it still does not provide external validation. MAE is interpretable in passengers/hour; RMSE penalizes larger residuals more; R-squared is relative to test-target variance and may be inflated by smooth synthetic patterns.

6. Service-planning and routing logic

Capacity and headway

The optimizer combines hourly estimated demand with configured train capacity, headway, fleet, and comfort assumptions. It calculates line recommendations and labels utilization/crowding bands. This is a scenario calculation, not live train occupancy, onboard load, or a verified DMRC timetable. At an interchange, line-demand allocation is an estimated split, not passenger tap-in/tap-out evidence.

Do not claim the model knows how many people are already on an arriving train, board at the selected station, or alight there. The current prototype does not model train-by-train passenger loads, origin-destination flows, or observed alighting/boarding counts.

Dijkstra travel times

`travel_graph.py` uses a custom adjacency structure and `heapq` priority queue; it does not need NetworkX. Nodes are `(station, line)` pairs so the graph can apply a transfer penalty when changing lines. Adjacent-station edges use coordinate-derived travel estimates when coordinates look plausible; missing, repeated-placeholder, or implausible coordinates use a configured default segment time. Dijkstra produces a shortest estimated time on this modeled graph. It is not a scheduled itinerary and does not include verified wait times, real-time disruption, or timetable changes.

7. Technology and design decisions

Technology	Project use	Why it fits now	Trade-off / next step
FastAPI	ASGI app, request routing, OpenAPI docs	API-oriented Python backend and learning goal	Explicit request/response schemas and dependency injection are not yet fully used
Uvicorn	ASGI server locally and on Render	Serves the FastAPI ASGI app	Worker/process sizing needs measurement against RAM because each process may load model data
Jinja2 + HTML/CSS/JavaScript	Landing, dashboard, report, charts	Keeps the current browser experience and simple server-rendered pages	API and templates share one app; larger UI may later separate
Pandas + NumPy	Data cleaning, generation, feature frames, numerical calculations	Standard tabular and numerical tools in the pipeline	Large CSV generation/training can be memory-heavy and remains offline
scikit-learn	HistGradientBoostingRegressor and Ridge baseline	Saved estimator works naturally with Python feature pipeline	Synthetic target validity is the dominant modeling limitation
joblib	Persist estimator and metadata	Convenient for trusted local Python model artifacts	Version compatibility and artifact trust must be managed
CSV/JSON	Station/operations data, synthetic data, metrics	Lightweight prototype inputs with no database	Provenance, coordinated updates, concurrent edits, and schema migrations are weak
heapq + custom graph code	Dijkstra route estimates	Small network and explicit transfer/edge assumptions	Topology/coordinates and default edge times need source verification
Git/GitHub + Render	Version control and deployment	Simple repository-based deployment	Deployment needs updated dependencies, start command, and a successful new build

Why FastAPI fits the current learning goal

The project has existing JSON endpoints, and FastAPI makes request/response contracts and interactive OpenAPI documentation central to the backend workflow. The migration preserves Jinja pages while enabling typed API models as a next step. The reason to migrate is the user's learning objective and API design practice, not a claim that FastAPI automatically improves model latency. The actual model/graph calculations are synchronous and use thread-pool execution.

8. Interview questions and strong answers

1. What does MetroInsights do?

Short answer: It explores station/date/time passenger-demand estimates and their relationship to configurable metro service scenarios.

Strong answer: “A user selects a station and scenario. FastAPI coordinates a saved synthetic-data-trained model, date/service context, a deterministic capacity/headway optimizer, and shortest-path estimates from a project-defined network graph. The dashboard visualizes the results. It is a planning prototype, not a live

feed or validated ridership forecast.”

Evidence: `app/routes.py`, `app/forecasting.py`, `app/scheduling_optimizer.py`, `app/travel_graph.py`.

Follow-up: What is the largest limitation? Answer: Training and evaluation targets are synthetic and operating/topology assumptions are not fully verified; observed passenger data and domain validation are needed before operational use.

2. Why did you migrate from Flask to FastAPI?

Short answer: The project owner wants to learn FastAPI, and its API-first design fits the existing JSON workflow.

Strong answer: “The existing pages and ML modules did not need to be replaced. I changed the app construction, route declarations, request parsing, template/static integration, dependencies, and Uvicorn launch configuration while keeping the response behavior. FastAPI provides OpenAPI docs and a path toward typed request/response schemas. The migration is a learning and API design choice, not a measured performance claim.”

Evidence: `app/__init__.py`, `app/routes.py`, `requirements.txt`, `Procfile`, `run.py`.

Follow-up: Is the migration runtime-verified? Answer: “The source passed Python syntax compilation and diff whitespace checks, but my local virtual environment did not have FastAPI/Uvicorn and package installation could not reach the index. I need to install requirements and run the local smoke check before claiming runtime verification.”

3. Why is the prediction route async if model inference is synchronous?

Short answer: Async is used to read the HTTP request body; inference runs in a thread pool.

Strong answer: “Pandas and scikit-learn calls are synchronous. I parse the request asynchronously, then call the synchronous prediction function through `run_in_threadpool`, which avoids holding the event loop during that work. Async syntax alone would not make CPU-bound work nonblocking.”

Evidence: `api_predict` and `_predict_data` in `app/routes.py`.

Common mistake: Saying async/await makes model prediction faster.

4. What does FastAPI add over a minimal Flask app here?

Answer: ASGI request handling, integrated OpenAPI documentation, and natural support for typed request/response schemas. The present API still parses a raw Request; adding Pydantic models is a future improvement. FastAPI was selected for the user's learning goal, not after a benchmark.

5. Where does the model get loaded?

Answer: `app/forecasting.py` loads the joblib estimator and metadata during module import. The web request uses those fitted artifacts; it does not retrain on the multi-million-row CSV. The trade-off is faster reuse after startup but startup can fail if artifacts are missing or incompatible.

6. How did you avoid leakage?

Answer: Pipeline 03 splits chronologically at 2026-10-01 and uses prior-date history aggregates. This reduces direct future-information leakage in the synthetic experiment. It does not make the labels real, eliminate every feature-construction risk, or validate performance on actual ridership.

7. What do the reported metrics mean?

Answer: On a 438,615-row synthetic chronological holdout, MAE is 26.58 passengers/hour, RMSE 51.10 passengers/hour, and R-squared 0.9946. These measure error against generated labels. I would not call them real-world accuracy or say “99.46% accurate.”

8. What is capacity utilization?

Answer: In the prototype it is modeled hourly demand divided by modeled comfortable boarding-clearance capacity for the selected scenario. It is not observed occupancy, people onboard, nor a train-by-train load estimate. The app lacks passenger alighting/boarding flows and current train loads.

9. How does the routing work?

Answer: The project builds a weighted line-expanded graph with `(station, line)` nodes and uses a heap-based Dijkstra search. Transfer penalties apply at modeled interchanges; segment times come from coordinates when plausible or a default assumption. The result is an estimated shortest time, not a scheduled itinerary.

10. Why keep synthetic CSV out of normal request handling?

Answer: The CSV is used for offline training/evaluation. Serving loads the saved estimator, metadata, and compact station/operations inputs. This avoids parsing millions of records for each page/API request. If retraining is required, the dataset must be provided or regenerated in a suitable build/data workflow.

11. What happens when the metro is closed?

Answer: Operating-hours logic marks the scenario closed, and the selected-hour modeled demand and train service are zeroed in the API response. Full-day profile values can still describe the operating-day curve, so I would distinguish the selected closed time from the profile chart.

12. What would you improve next?

Answer: First complete runtime smoke checks and add endpoint tests. Then define Pydantic input/output models, deliberate 404/422 behavior for unknown stations, structured logs, and a health check. For model validity, obtain observed station/time ridership and verify topology and operations assumptions before using the forecast for decisions.

13. How would you scale it?

Answer: Measure latency, CPU, and memory first. Keep the API stateless; use a compatible Uvicorn worker/process strategy within memory limits, since each process may load its own model. Cache graph/profile data if measurements show repeat work. Move retraining to offline jobs. Add queues or separate model serving only if measured workload warrants their operational complexity.

14. What are the security concerns?

Answer: The app has no authentication/authorization or rate limits, so it is intended as a public demo, not a privileged operations console. Inputs should have explicit schemas, station membership validation, sensible date bounds, payload limits, and exception-safe messages. Joblib files must be trusted because deserialization can execute code. Keep secrets out of Git and dependencies updated.

15. What have you personally done?

Describe only work you actually performed. A safe structure is: “I migrated the web layer from Flask to FastAPI by changing app construction, routes, request parsing, template/static serving, dependency declarations, and Uvicorn deployment startup. I preserved the forecasting, scheduling, and routing domain logic. Runtime verification is pending installation of the new dependencies in my local environment.” Do not claim deployment or tests that have not happened.

9. Failure scenarios and practical checks

Scenario	Likely symptom	Check / mitigation
FastAPI/Uvicorn absent	`ModuleNotFoundError` at app import	Install from updated `requirements.txt` in the active environment
Missing/incompatible model artifact	Import/startup failure in forecasting module	Verify committed artifacts and pin compatible Python/scikit-learn versions
Missing multipart parser	Form POST fails parsing	Keep `python-multipart` in requirements
Bad date/time	API 400 response	Test valid and malformed date/time boundaries
Unknown station	Downstream lookup error or 500	Validate against station directory and return a deliberate client error
Empty/invalid JSON	400 error for malformed/non-object body	Add tests for empty, array, null, and malformed JSON
Stale frontend response	Older prediction may overwrite a newer one	Existing JavaScript request ID guards stale updates; preserve it
Bad coordinates or incomplete topology	Implausible or fallback travel time	Record edge provenance, validate stations and topology from authoritative sources
Render still serves old release	UI/framework unchanged	Push migration commit and wait for successful dependency install/build/deploy
Too many workers for RAM	OOM/restarts	Measure per-worker model/data memory before increasing process count

No destructive security testing is needed. Verify input/output handling in a local development environment and never flood the public service.

10. Learning roadmap and exercises

Essential sequence

1. HTTP and ASGI basics: methods, status codes, JSON/form bodies, request/response lifecycle, ASGI server vs framework.
2. FastAPI routing: `FastAPI`, `Request`, path operations, dependencies, error responses, OpenAPI.
3. Pydantic: typed inputs, validation errors, response models; refactor `/api/predict` after understanding the current raw Request flow.
4. Jinja/static: template context, request-aware static URLs, HTML escaping, frontend fetch contract.
5. Concurrency: sync vs async, event loop, thread pool, CPU-bound vs I/O-bound work.
6. Model serving: feature parity, artifact versions, startup loading, inference latency, data provenance.
7. Testing and observability: TestClient/HTTPX, route contract tests, logs, health/readiness, dependency reproducibility.
8. Deployment: Uvicorn command, `$PORT`, build logs, memory, restarts, deploy rollback.

Exercises

- Trace one browser selection from `dashboard.js` to `/api/predict` and back to a chart/KPI.
- Add a Pydantic `PredictionRequest` with station/date/time/line fields and verify invalid values return 422 with a useful message.
- Add deliberate unknown-station handling and ensure both HTML form and JSON API respond safely.
- Add a health endpoint that checks app process readiness without loading/retraining the large CSV.
- Measure cold-start and prediction time separately; do not infer speed from framework choice.

- Create a small fixture for closed-hour boundaries (04:59, 05:00, 23:29, 23:30) and verify expected service flags.
- Verify model feature names/order at serving time against the saved metadata.
- Mock inference and check response keys/units rather than asserting the model's synthetic output is operational truth.

11. Mock interview plans

30-minute interview

Minutes	Segment
0-3	Project intro and intended user
3-8	FastAPI architecture and request flow
8-14	Model, 23 features, chronological synthetic evaluation
14-19	Scheduling assumptions and capacity meaning
19-23	Dijkstra graph and data gaps
23-27	Failure/debugging and FastAPI migration status
27-30	Limitations and next steps

45-minute interview

Use the 30-minute flow, then add a live walkthrough of `app/routes.py`, schema/error handling design, and a design discussion on observed ridership and train-level load modeling.

60-minute interview

Add a 10-minute system-design exercise: “Design the next version using real station-hour counts, safe forecasting, and auditable recommendations.” Ask about data contracts, time-based validation, privacy, model versioning, cache boundaries, monitoring, fallback behavior, and staged rollout. Do not over-engineer with microservices before traffic and failure requirements are known.

Interviewer scorecard

Score 1-5 for each: clear problem explanation; accurate FastAPI/ASGI understanding; evidence-based code walkthrough; correct synthetic-metric caveat; clear distinction between demand and train capacity; honest limitations; sensible testing/deployment plan; concise communication. A high score requires precision and honesty more than inflated claims.

12. Final quick-reference sheet

Keep these facts straight

- 243 modeled station records; 242 reachable destinations from Rajiv Chowk in the displayed graph result.
- 3,361,176 synthetic rows = 243 stations x 728 dates x 19 hourly slots.
- 23 features, 438,615 chronological synthetic holdout rows.
- MAE 26.58, RMSE 51.10, R-squared 0.9946 on synthetic targets only.
- FastAPI app, Jinja2 templates, static HTML/CSS/JS, Uvicorn ASGI server.
- Custom heapq Dijkstra graph; do not claim NetworkX.
- No database, live DMRC data feed, observed ridership validation, or train-by-train occupancy model.
- Runtime smoke check is still pending installation of FastAPI/Uvicorn in the existing local virtual environment.

Avoid these claims

- “The model is 99.46% accurate on real Delhi Metro ridership.”
- “Capacity utilization tells us how many passengers are currently onboard.”
- “All travel times are official/timetable-accurate.”
- “FastAPI made model inference faster.”
- “The project currently uses NetworkX.”
- “The migrated app is deployed and verified” unless you have completed a successful build and checked the live service.

One-line close

“MetroInsights demonstrates an end-to-end synthetic demand-planning workflow with a FastAPI web layer, saved scikit-learn inference, transparent scheduling assumptions, and topology-based route estimates, while clearly identifying what still needs real data and operational validation.”

Repository evidence map

- App construction and static mount: `app/__init__.py`
- Uvicorn local startup: `run.py`
- Render start command: `Procfile`
- Dependencies: `requirements.txt`
- Routes/request flow: `app/routes.py`
- Feature assembly and inference: `app/forecasting.py`
- Training split/features/model metrics: `pipeline/03_train_and_evaluate_forecaster.py`, `models/evaluation_metrics.json`
- Synthetic scenario generator: `pipeline/02_generate_calibrated_dataset.py`
- Data and service assumptions: `data/cleaned/`
- Scheduling: `app/scheduling_optimizer.py`
- Graph and topology: `app/travel_graph.py`, `app/network_topology.py`
- Browser API contract: `app/static/js/dashboard.js`